Update **confidential** designator here

OpenShift Virtualization

Technical overview: networking stack

Platform team

Networking deep dive

Red Hat

Update **confidential** designator here

Agenda

Session outline

- Architecture: VM, pod, CNI data path
- Primary network: OVN-K and virt-launcher
- Multus, NAD, bridge, and IPAM
- CRDs, live migration, SR-IOV, policies
- Operations: diagnostics and common pitfalls

As of: OCP 4.x with OVN-Kubernetes

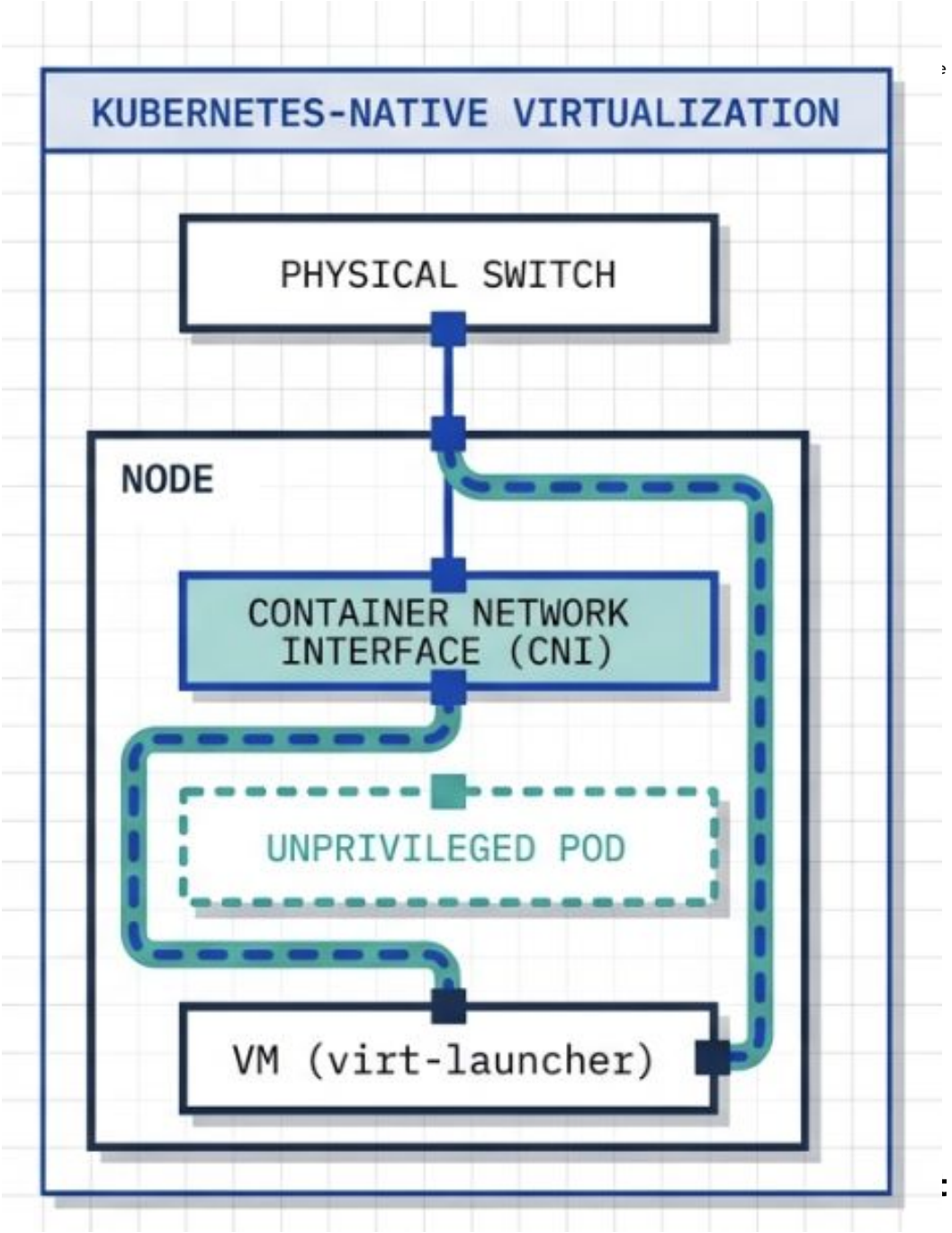
Update **confidential** designator here

Learning objectives

Takeaways

- Name the network endpoint of a VM
- Place Multus NADs in the design
- Plan migration and MTU deliberately
- Weigh hardware vs. virtual NICs

Architecture



Update **confidential** designator here

End-to-end data path

From guest to switch

- Guest: virtio-net or PCI passthrough
- QEMU inside the virt-launcher pod
- CNI interfaces on the pod
- OVN or bridge path on the node

Source: KubeVirt / OCP networking

Update **confidential** designator here

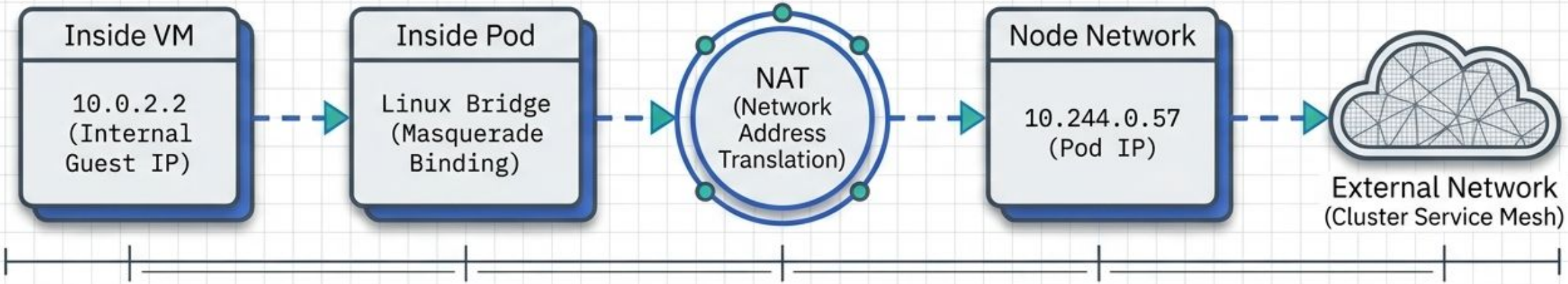
virt-launcher and libvirt

One pod per running VM

- libvirt drives the VM lifecycle
- Tap or veth attachment to pod network
- Resource limits apply like any pod
- Metrics and logs via the pod surface

Primary CNI

Layer 1: The Default Pod Fabric via Masquerade



Architectural Constraint: Because traffic is masked behind the ephemeral Pod IP, persistent connections drop during Live Migration.

Update **confidential** designator here

OVN-Kubernetes for VM pods

Same model as application pods

- Logical switch port per pod interface
- Geneve overlay between worker nodes
- NetworkPolicy applies to the VM pod IP
- Cluster DNS and services work unchanged

Update **confidential** designator here

Primary network in context

What overlay gives you—and what it does not

- Consistent pod IP per namespace
- East/west across logical switches
- North/south via gateway paths
- Simpler ops without extra VLANs
- L2 segmentation often needs Multus
- Account for MTU overhead
- Broadcast domains limited in overlay
- Performance tuning: queues, NUMA

Multus & secondary networks

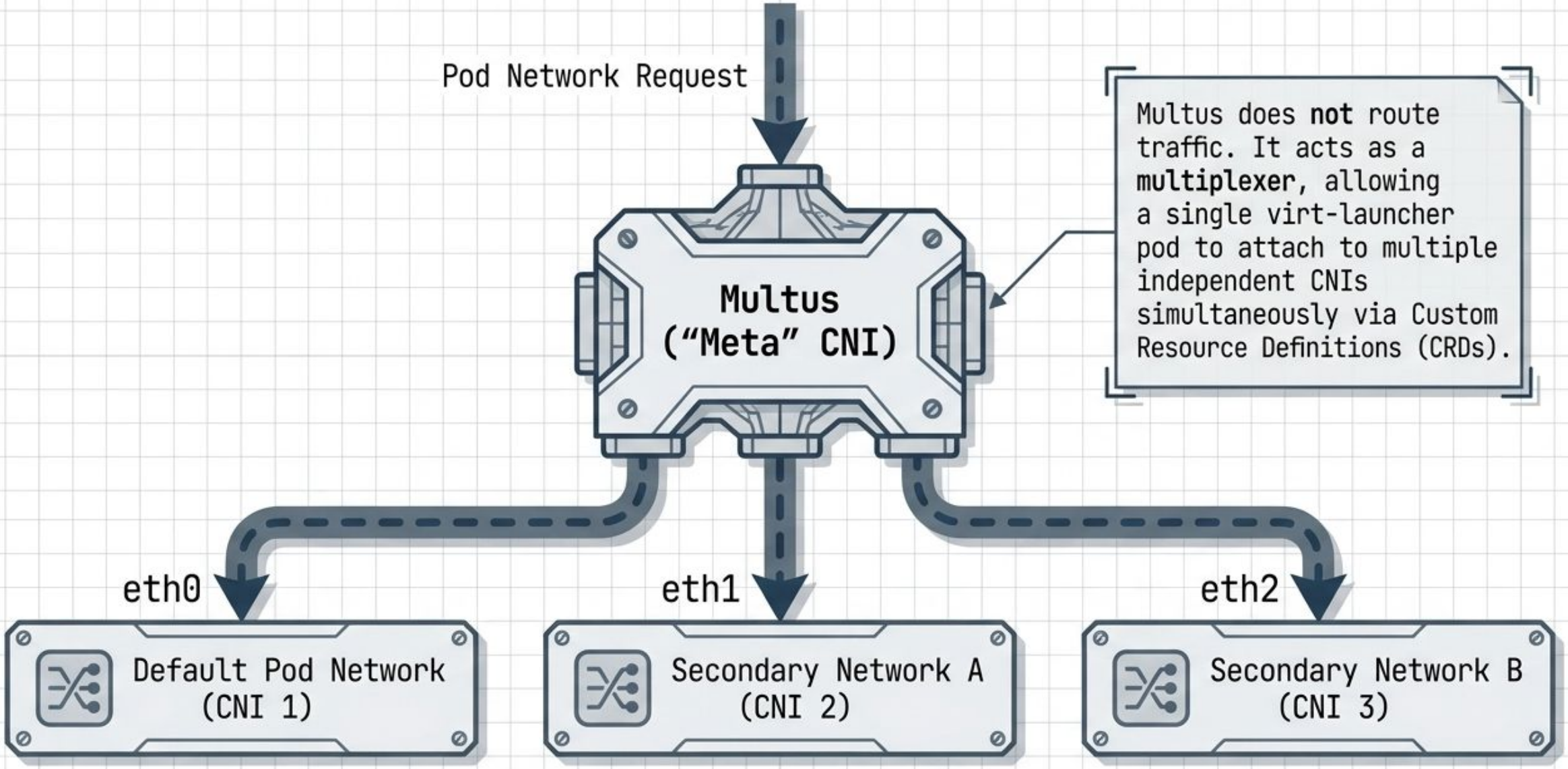
Update **confidential** designator here

NetworkAttachmentDefinition

Declarative CNI JSON

- CR kind NetworkAttachmentDefinition
- CNI type reference in config JSON
- Namespace-local or shared use
- Multus chained invocation after primary CNI

MULTUS: THE CNI MULTIPLEXER MANIFOLD



Update **confidential** designator here

Bridge CNI and host bridges

L2 attachment to infrastructure

- Linux bridge on the worker node
- VLAN IDs definable in NAD JSON
- Consistent MTU: guest, bridge, ToR
- Align redundancy and STP with network team

Red Hat

Requires infrastructure alignment

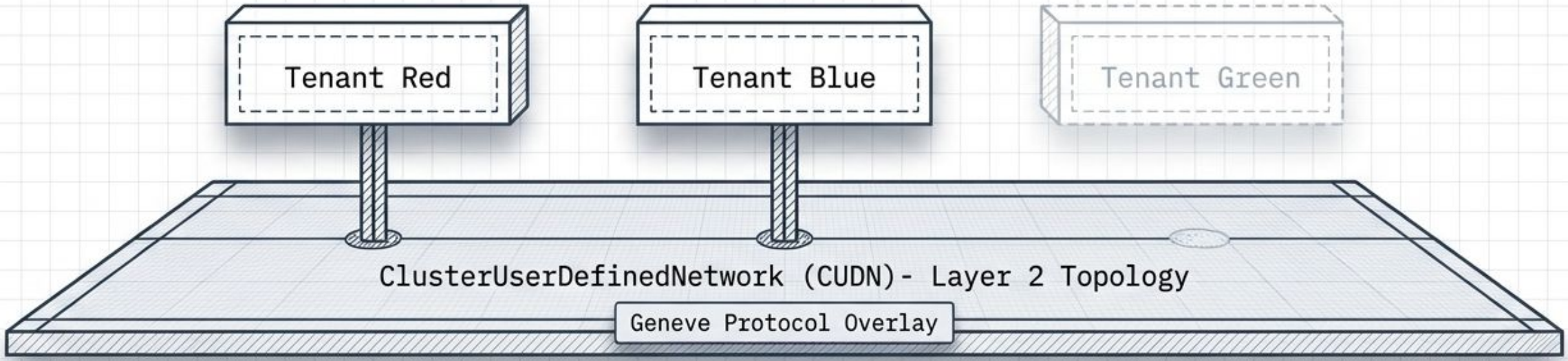
Update **confidential** designator here

IPAM for secondary interfaces

Whereabouts, host-local, static

- Whereabouts uses Kubernetes leases
- Plan pools per VLAN or per team
- Avoid overlapping CIDRs
- Pin MAC addresses when required

Layer 2: Achieving Tenant Isolation with Primary UDNs



Native Tenant Isolation
(Replaces default pod network without complex Network Policies).

Persistent IP Addresses
(IPAM lifecycle: Persistent).

Seamless Live Migration
(No NAT required)

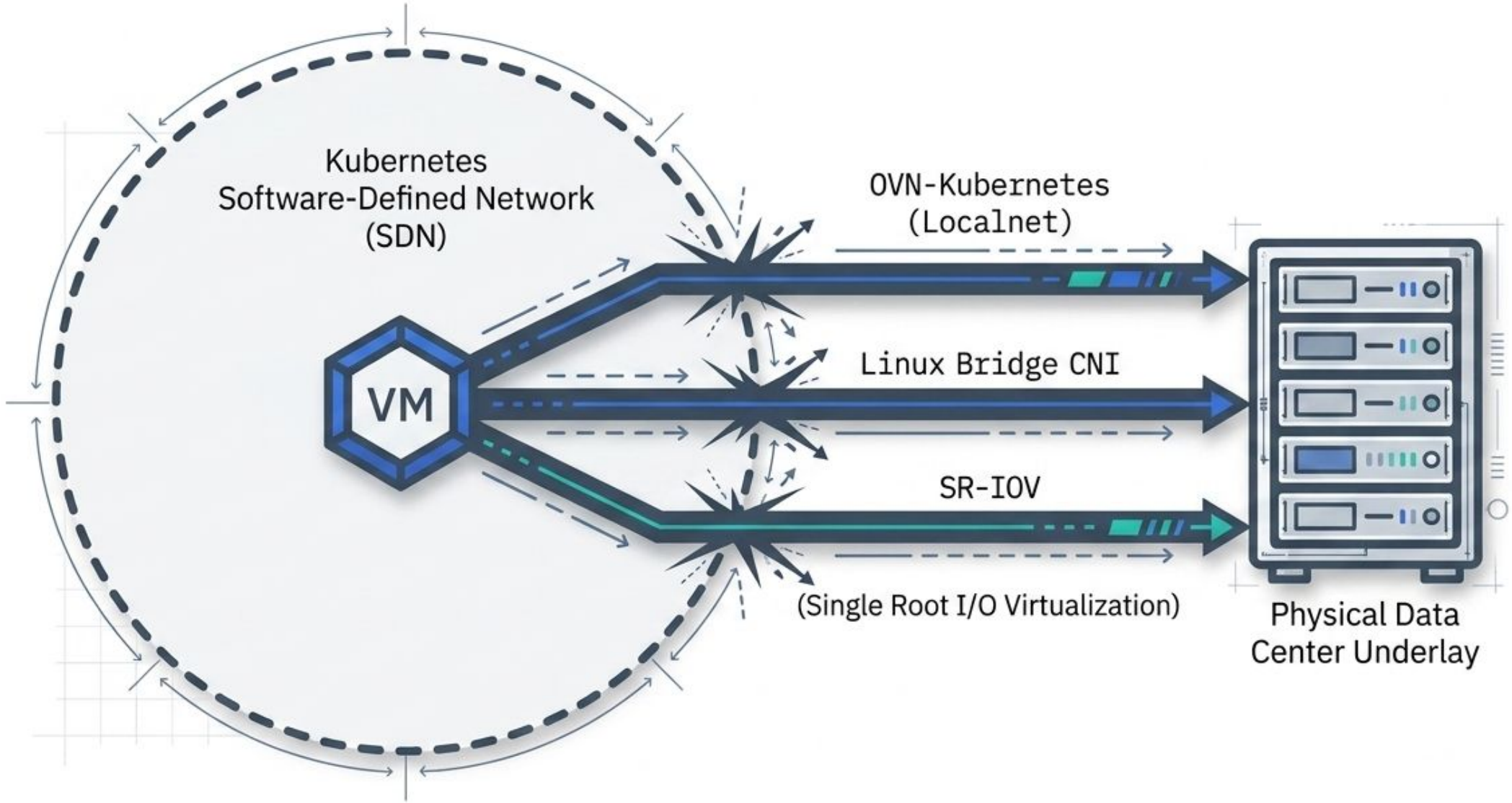
NotebookLM

Paradigm Shift: Masquerade vs. Primary UDN

Architectural Dimension	Default Pod (Masquerade)	Primary UDN (Layer 2)
Translation Layer	Network Address Translation (NAT)	Native L2 Overlay (Geneve)
IP Persistence	Ephemeral (Changes on reboot/migration)	Persistent (Retained across lifecycle)
Live Migration	Connections interrupted	Seamless state transfer
Security/Isolation	Requires MultiNetworkPolicy	Native namespace-scoped isolation
Primary Use Case	Standard cloud-native microservices	Legacy stateful VM workloads

NotebookLM

Layer 3: Piercing the Overlay (Secondary Networks)



NotebookLM

APIs & live migration

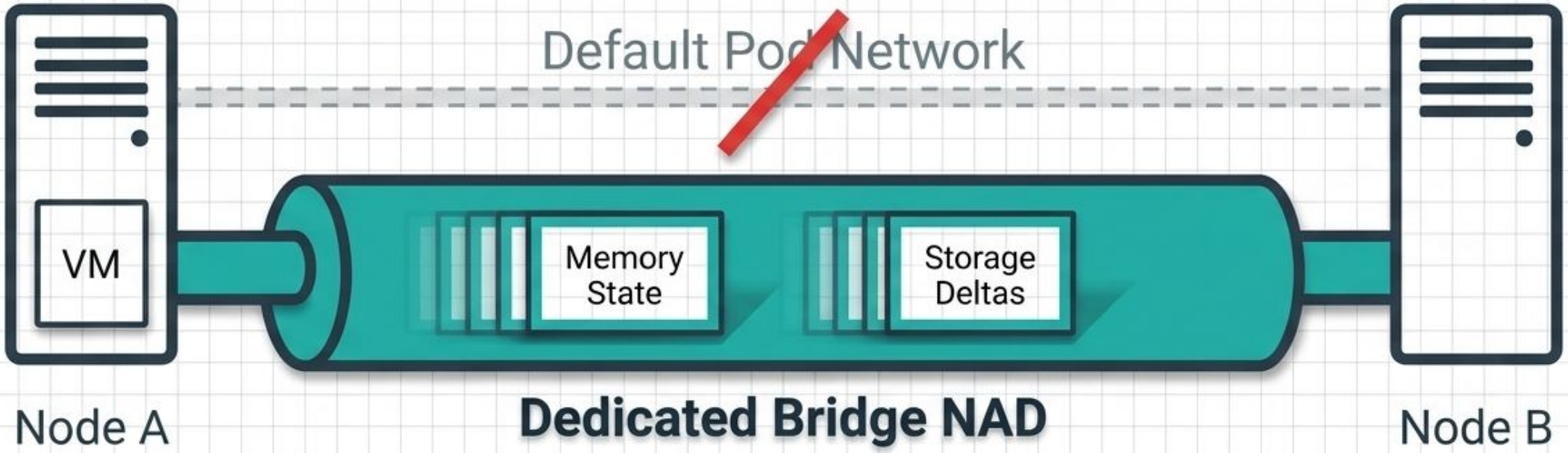
Update **confidential** designator here

VM networking in CRDs

VirtualMachine and network list

- Networks per VM referenced in spec
- Type: podNetwork or multus network name
- Status shows bound interfaces
- Cloud-init or NMState extends guest config

Operational Reality: Dedicated Live Migration Networks



Configuring a dedicated secondary bridge network in the `HyperConverged` CR isolates migration traffic, preventing control-plane saturation during maintenance events.

NotebookLM

Live migration: networking

Source and destination must match

- Same NADs and VLAN IDs on both sides
- Stable IPs only with correct L2 design
- Bandwidth and latency of migration path
- Timeouts on slow secondary networks

Update **confidential** designator here

virtio-net vs. SR-IOV

Trade-offs at a glance

- virtio: flexible, migratable
- Standard guest drivers
- Scale with multiqueue
- Lower latency than NAT-only paths

- SR-IOV: very low latency
- VF directly in the guest
- Operator: SriovNetworkNodePolicy
- Migration usually limited

Physical Underlay Diagnostic Matrix

Dimension	OVN-K (Localnet)	Linux Bridge CNI	SR-IOV
Host Dependency	Open vSwitch (OVS) Config	NMState Operator	SR-IOV Operator & Hardware
Network Policies	Supported (MultiNetworkPolicy)	Not Supported	Not Supported
MAC Spoof Filtering	Always On	Configurable	Hardware Dependent
Data Plane / Performance	Standard	Standard	High Bandwidth / Low Latency (DPDK)
Live Migration	Fully Supported	Fully Supported	Highly Restricted (Hardware limits)

NotebookLM

Update **confidential** designator here

Kubernetes networking features

Services, policies, edge

- ClusterIP targets the virt-launcher pod
- Routes and ingress same as for pods
- Apply NetworkPolicy on pod labels
- EgressIP and EgressFirewall per design

Version number here V00000

Update **confidential** designator here

Diagnostics and common failures

Narrow issues quickly

- oc get vmi,pod -o wide for phase and node
- virt-launcher events for CNI errors
- Verify MTU chain from guest to ToR
- Node: ip link, bridge, VF state

Thank you

Documentation: OpenShift Virtualization |
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virtualization/

Networking overview |
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/networking/

linkedin.com/company/red-hat

youtube.com/user/RedHatVideos

facebook.com/redhatinc

twitter.com/RedHat

Red Hat